

Final Year Project Report

Full Unit – Final Report

Star Student App – Building a Mobile Application

Leane Matejko

A report submitted in part fulfilment of the degree of

BSc (Hons) in Computer Science

Supervisor: Susnas Sourjah



Department of Computer Science
Royal Holloway, University of London

Student's first and last name, Year

December 19, 2025

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 9737

Student Name: Leane Matejko

Date of Submission: 19/12/2025

Signature:

A handwritten signature in black ink, appearing to read 'Leane Matejko', with a large, stylized flourish extending to the right.

1 Abstract

Starting university or leaving secondary education is typically a moment of high stress and development for young people as they are learning to navigate the world without the structure of most secondary education systems. There is a significant decrease in direction given to students and for most this is the first time that they will be living alone and away from family. This can lead to many students' developing poor mental health and struggle to find effective coping mechanisms that can support them[2]. Student mental health services have seen an increase in students requiring support and an increase of students' report feeling depressed, anxious and experiencing high levels of stress[3]. Many contribute these feelings towards stressors such as financial inexperience, educational demands and poor organisational skills. Many platforms require significant input from users, are difficult to set up or subscriptions are needed to be paid for them to be effective.

If someone were to already be overwhelmed, facing financial insecurity or unable to dedicate time to set up the platform, this ultimately negates the point of service. Instead, I am going to provide an app to users where they can stay organised and learn effective life skills while not being burdened.

To achieve this, I am going to develop my app 'Star Student' that introduces easily accessible progress feedback areas, promoting good lifestyle habits, and understanding how certain actions can lead to burnout. Some healthy levels of stress can help promote better long-term memories and support the learning of students, so encouragement to try new skills and techniques can be added through memory logs and achievements[1]. We can also monitor to track these same actions and see where harmful habits are created, for example, students studying for extreme periods of time without breaks. I hope to achieve this by using GPS features to allow for 'study spaces' to be determined by users, allowing them to automatically track study periods but also be alerted when to take a break. This will be done through a combination of the user's GPS tracker and a timer that will log a session if the user is within a saved 'study space' for longer than 5 minutes. Libraries and other common studying spaces can be included if users do not want to create their own 'study spaces'.

Users will also have access to smart planners that can have automatic and manual entries added to help them with possible deadlines. These can be synced with pre-existing calendars and used in conjunction with the journal log. To support reflection and consistency, users will have the option to include a daily log in an audio or written format.

Photos taken throughout the day can be included as well, with an option to see where they were taken on a map. And at the end of each week, a summary of their achievements and progress will be given, further enforcing those positive habits. Users will also have the option to track their budgeting via smart categories and alerts once thresholds are reached, with an emphasis on visual representations to help convey growth and possible improvements.

Market research and testing will also be carried out throughout to see how users utilise the software, and what features of the UI are easily understandable. This should be an app for beginners and should be easy to approach. This app will be developed using the Android Studio IDE using the Kotlin programming language. Testing will be done through the built-in testing platform in Android Studio, using a TDD approach to ensure all features work as expected. There will be a round of further security and functional testing completed to ensure users are able to use the app safely.

Figma has been used for UI design and visual planning. Jetpack compose was used to develop the UI layer and components, and Firestore/Firebase (cloud-based relational database system) was used for the current database connection. Future versions will likely utilise local database options for the storage of personal information, however a suitable one needs to be found. Brevo is being used in conjunction with some of the firebase APIs to allow for verification emails to be sent to users for sign up. And finally, Android Studio is the main IDE used for the general development, testing and provides emulators to create the UI renders and behaviours.

Overall, while there exist platforms for students and people in their early careers, I hope to provide an option to help beginners make decisions to support their health and learn new things. Existing platforms can be future options after this app as it could be more customisable, as this app should help users find works best for them and let them easily keep track of their progress.

2 Introduction

2.1 Motivation for the project

Many of my motivations to create this project relate back to the difficulties I had when organising myself once I had left sixth form. Most of the structure and organisational skills I had started with made it very difficult to adjust to the new environment and it took a significant amount of time to find a system that works for me. The fast pace and stacking deadlines lead to high amounts of stress and poor lifestyle choice that ultimately resulted in me needing long recovery periods once terms were completed and a general sense of unrest and a lack of confidence in my overall abilities. In study, it is discussed that many students struggle to self-regulate and be able to adhere to the goals set by themselves, and I would like this project to act as another layer of support to them[8].

Therefore, I wanted to build an organisation app that requires minimal setup that allows for my progress to be recorded and represented in an easy-to-understand way that is encouraging.

There are several studies that show that reasonable amounts of stress can support learning and provide students the chance to push themselves and see results[1][2], however requiring the users to continually fill out information can make the app counterproductive. Therefore, having some semi-hands-off features could help with the overall task fatigue that students are facing, potentially developing some good coping skills.

2.2 Technology choices

Here I will outline some of the technology choices that have been made for this project.

2.2.1 Android Studio

Android studio provides an IDE that greatly supports app development on many platforms, even outside of android native. There are many integrations with Google APIs and services that allow for everything to be more easily added to the project, alongside a massive history of documentation and tutorials for guidance on what to do next.

Not only this, but there are several emulators and display menus that can be used to support UI development and overall functionality testing. While not all features are readily available, there are many dependencies and add-ons that can be used when such pitfalls arise (for example, there is a lack of UI coverage awareness - > Jacoco dependency can be easily added).

Builds can be easier compiled and run, with service levels of logs that are shown through Logcat, which can be tagged and filtered for specific results, all of which is critical for debugging and finding logical errors.

2.2.2 Firestore/ Firebase

Firestore or Firebase is NoSQL cloud-based database platform created by Google that has multiple features that support scalability, flexibility and real-time responsiveness[9].

While I would like to move away from storing user's information within the cloud, Firestore was one of the best options to add database connectivity to the project currently. Android Studio and Firestore have a pre-existing integration, so once the authentication keys were correctly set, several database actions could take place and be tested.

Firestore also facilitates other cloud functions from Google Cloud that can be used for REST APIs that might be triggered in the background.

While Firestore has several other features, currently it will be used to store users' credentials for signing in and storing registered user's information.

2.2.3 Brevo/NodeMailer

To have the email notifications that would be used for registering or alerts, an SMTP platform is required to act as a middleman between the app and users. By using Brevo or similar platforms, this prevents developers from potentially spamming their users with email, as there is a general limit to the amount of traffic that a single person can send out.

While there are further features related to SMS and WhatsApp, I am going to instead focus on the email integration first.

2.2.4 Figma

Figma is a UX/UI designing platform used by many industry professors to create semi-working prototypes that produce a similar feel to the final product. These prototypes can even be given to users and exported in many formats that allows for a feedback/development cycle that is closer to the user's requirements.

Figma also has many features relating to android and general development, allowing for themes, designs and colours to be easily transferred over.

There are also some version control features via the program's version history, allowing for work to be reliability stores and can be accessed from multiple devices.

2.3 List of Key Milestones (Timeline)

2.3.1 Low fidelity Prototype

Expected Completion: Week 3, Actual Completion: Week 5

Goal: Create a low fidelity prototype that outlines the functions of the program.

Progress: Complete.

Commit Proof:

- 6ea95a5c Added Figma prototype and images.

2.3.2 Database Connection

Expected Completion: Week 4, Actual Completion: Week 8

Goal: Add database read and write capabilities to the app, accessible via the UI.

Progress: Complete.

Commit Proof:

- 4f916051 Added functionality and testing for internet connectivity.
- 43468bf6 (database) Added Database class and added some of the architectural organisation.
- 843e2ac8 Updated NetworkConnectivity function to be compatible with SendEmail.

2.3.3 Attempts to set up a CI/CD pipeline

Expected Completion: - Week 3, Actual Completion: N/A

Goal: Add a CI/CD pipeline for tests after pushing to GitLab.

Progress: Uncomplete/blocked.

Commit Proof:

- 61701a9d Added ci yaml and dependencies.
- 4c4fb47a Added ci yaml and dependencies.
- b7299c15 Updated permissions within gitlab-ci.yaml
- 86c56bc7 Updated gitlab-ci.yaml to fix the pipeline.
- cb1c0318 Updated gitlab-ci.yaml to fix the pipeline.
- 4d3a58f9 Updated gitlab-ci.yaml to fix the pipeline.
- c9e09fe0 Updated gitlab-ci.yaml to fix the pipeline.
- f67e8b7d Updated gitlab-ci.yaml to fix the pipeline.
- e5f485ce Updated gitlab-ci.yaml to avoid permission issues.
- 19ff15cd Updated gitlab-ci.yaml to avoid permission issues.
- 80d958f9 Updated gitlab-ci.yaml to avoid file location issues.
- 9ea4e8ac Updated gitlab-ci.yaml to avoid file location issues.
- 738e6259 Updated gitlab-ci.yaml to avoid file location issue.
- c8cd6072 Simplifying pipeline yaml to only run unit tests.
- 933e8198 Attempting a generated solution for troubleshooting.
- 744acec2 Updated docker image used.
- 65764b9d Updated docker image used.
- 12cb80e7 Removing the mkdir command from the ci yaml avoids the pipeline failure.
- cb71e134 Regressed to simplified ci yaml.
- 5f2a03fa Added missing navigation.
- 20cae72f Added missing dashes to ci yaml.
- 1072f332 Added SDK for CI.
- 0c99e94b Added missing variables to ci yaml.
- c03e7cd8 Removed everything related to the pipeline -> will readdress later.

2.3.4 UI Components

Expected Completion: Week 3, **Actual Completion:** Week 5

Goal: Create the UI components to be populated throughout the app's screens.

Progress: Complete.

Commit Proof:

- 2dafb6c6 Added background component
- 2a34e7dc Added medium icon component
- 861492ca Added small progress Widget
- 208f31cc Added a large navigation widget.
- 4a7d212f Added avatar windows and reorganised some of the large Navigation widgets to be more generic.
- 9db70c15 Added text field component
- cb064e32 Added text input component
- 17dd1344 (origin/initialUI, initialUI) Added button component.

2.3.5 View Screens

Expected Completion: Week 4, Actual Completion: Week 9

Goal: Create the basic UI screens that are shown to the user.

Progress: Complete, with further development in Term 2.

Commit Proof:

- 1634355d Implemented some basic UI navigation from the opening page to the sign in (existing users) page.
- a7ad3aa4 Added UI element for email check page.
- d433fdf4 Added missing password field for the sign in UI.
- b99d08a8 Added screens and navigation for the add email, verify email and basic homepage.
- 11fb3d7c Added UI elements and navigation for the verified email page.
- 0cbe11cd Added classes for ViewModels for all current screens.

2.3.6 Navigation

Expected Completion: Week 4, Actual Completion: Week 9

Goal: Add basic navigation through the app to the different screens.

Progress: Complete, with further development in Term 2.

Commit Proof:

- 1634355d Implemented some basic UI navigation from the opening page to the sign in (existing users) page.

2.3.7 Testing

Expected Completion: Week 9, Actual Completion: Week 11

Goal: Add unit and UI testing to the current versions of the code system.

Progress: Complete, with further development in Term 2.

Commit Proof:

- 9e75b9b4 Added tests for checkRealEmail().
- c9529480 Added UI test and dependencies. Test for avatarWindow
- a9185381 Added UI test for the background render.
- 06a09d21 Added UI test for the medium icon widget.
- 6a1cde84 Added UI tests for the small progress widget.
- d0179d6f Added UI tests for the large navigation widget and updated the tests for the avatar window.

- 7653d1c4 Added UI tests for the text field components.
- dc6fbfb3 Added UI tests for the spacer and input field components.
- 36043be4 Added UI tests for the button component.
- d6fb8f54 Fixed UI tests. Introduced the beginning of coroutines to ensure correct process flow.

2.3.8 Theory and Report

Expected Completion: Week 3, Actual Completion: Week 11-13

Goal: Complete research and report required for development.

Progress: Complete, with further development in Term 2.

Commit Proof:

- bfd43012 Added pdf references to my documents.

3 Backgrounds and Theory

3.1 Human Behaviour

The main aim of this app is to support university students during their studies; therefore, I completed some research ways to support good memory retention and be able to maintain good habits.

Stress management is incredibly important, and it is the long-term extreme forms of stress that can negatively affect people and where we see the bad side-effects. However, it is the short-term forms of stress where we can see growth and a significant percentage in progress. Utilising these ideas should be able to support learning and give students health levels of challenge. [1]

Students usually struggle attempting to uphold specific expectations that they had initially set up, along with an inability to see long term progress, can lead to poor coping skills and bad lifestyle habits to occur. Instead, we want to start consistent habits that tackle fragments parts of these goals and showcase the progress made to reinforce these habits.[8]

Introducing progress bars alongside point systems can help with students being able to see progress made, as well as periodic summaries of current actions. [18] However, this approach might not work for students who become easily overwhelmed with similar processes, therefore options to remove or hide these features will be made available.[7] While this would not completely cover every type of user, it broadens the scope of users that can find the ‘Star Student’ app useful.

Prompts through notifications and banner within the app will encourage recall activities for specific topics, to help solidify topics within long-term memory, avoiding the need for cramming and extreme study periods. [8]

3.2 Design Principles

As humans we are drawn towards good design and we more forgiving of the general process the more easily we can understand information. Below I have attached an example of how even simple principles such as high contrast, appropriate spacing and general consistency are important. All of this is especially important on the scale of a phone, as users have limited options to enlarge the UI or change it in a significant enough way to improve the general readability.

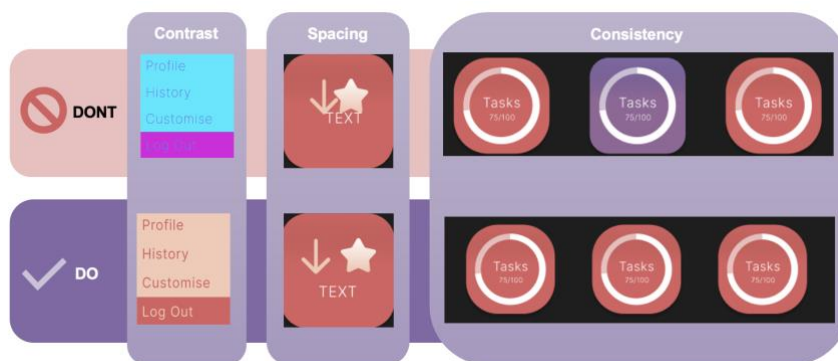


Figure 3.2.1 Examples of ineffective and effective contrast, spacing and consistency

Contrast within the hues and concentration of colours is important, as users with colour-blindness or similar visual impairments would be unable to see features of the UI (text, symbols, images...).

Like contrast, spacing is important as human's brains are wired to find implied patterns and create groups. Gestalt's psychology largely describes many of these concepts and how we process patterns. UI elements that are too close to each other are perceived to be a single unit, decreasing the overall readability. While in the example there are a combination of words and icons, this same principle could apply to other components such as text field or menus.

Consistency is also important. Similarly to how we need appropriate spacing, we might find an unexpected, applied pattern that makes us believe that a component has a different function when compared to its counterparts. I have tried to apply the concept that components of the same function should have the same look. This is why I have decided to create reusable UI components that can be added to the required screens.

3.3 Android App Lifecycle

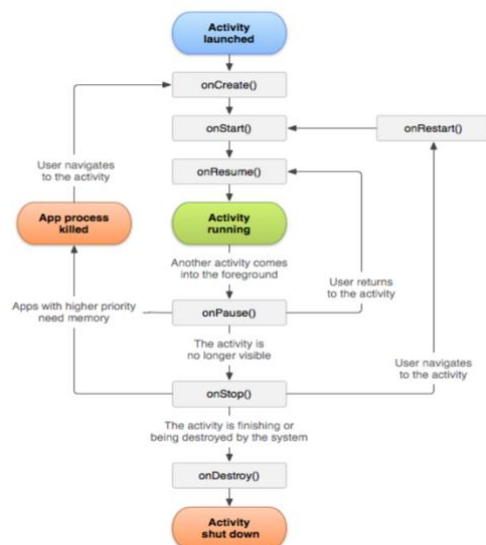


Figure 3.5.1 Simplified Diagram of the android

activity lifecycle [11]

The android app lifecycle dictates how your application behaves while the app is being directly interacted with and what happens in the background. The activity instances are responsible for transitioning between these different states and allow for callbacks whenever the state changes or the system is currently creating, resuming, stopping or destroying the activity or process.

You can declare how your app behaves once the user leaves or resumes an activity. Ensuring that your app can handle these changes in state is paramount as it introduces robustness and there to be expected level of performance. This allows for the other functions of your phone to not have a detrimental impact on the process flow and data to not be potentially lost.

The android activity lifecycle has six core callbacks:

Callback	Purpose	Example
onCreate()	Starts when the system first creates the activity. Begins the basic application logic and should only occur once for the entire lifecycle. The activity does not remain in this state for a long period, instead moving onto the onStart() and onResume() methods.	Binding the XML logic to the main activity
onStart()	Make the activity visible to the user and prepare the activity to enter the foreground.	Where the UI is initialised
onResume()	The activity can enter the foreground. This is where the app interacts with the user and main point of contact. Any lifecycle-aware components that are tied on_resume callback can enable any function needed while the component is visible. This function will be recalled every time that the resume state is returned to. It is possible for a component to be visible but not in a resume state, however the general process flow would need to be changed.	Get the camera component
onPause()	Called once the user is leaving your activity, doesn't always mean that it is being destroyed, instead the activity is moving away from the foreground. The lifecycle components can stop any functionality that does not need to be running the foreground, or any resources that can be released. Very brief execution, so any processes that require to be fully completed should not be within this method. Avoids the need to re-initialise components.	Opening a dialogue window
onStop()	The activity is no longer visible to the user and is likely about to be terminated. Any lifecycle-aware components can be stopped at any time and that are not visible on the screen. Any unneeded resources are released or adjusted. Used to shut down any CPU-intensive shutdown operations. Components still do not need to be reinitialised at this stage.	Saving to persistent storage
onDestroy()	The activity has been finished, or the activity has been temporarily destroyed due to a configuration change. The final lifecycle callback that the activity receives.	The app has been closed

Each of these callbacks cause the activity to enter a new state. Often, when the user switches between these states, the app's information is stored somewhere on the device so the activity can resume. We want to avoid the potential for ejection from memory, as then the app is unable to recover from killing the process, leading to possible permanent data loss.

The system may require that specific processes are killed to free RAM for future processing, however there is a general hierarchical structure to ensure the least important processes are destroyed first.

Here is that general structure (From the android developer's documentation):

Likelihood of being killed	Process State	Final activity state
Lowest	Foreground (having or about to get focus)	Resumed
Low	Visible (no focus)	Started/Paused
Higher	Background (invisible)	Stopped
Highest	Empty	Destroyed

Activities will always be protected against being killed to free memory; however, the processes would be the parts affected. Understanding how to store the UI state is important to maintaining consistent representation and having the ability to save and restore these states.

3.4 Android Processes, threads, worker threads and services

Processes are required by the lifecycle to ensure that the memory and system can continue the phone's function and all the app to run its own Linux process. The general project lifecycle should not be controlled directly by the application, instead in conjunction with the overall system. [16]

There are a variety of processes available to utilise within the code:

Process Name	Function
Foreground Process	Required for what the user is currently doing. Responsible for running the activity of the top screen that the user is currently interacting with and contains the Broadcast receiver for the method currently running.

	Limited in number Last of the processes in the hierarchy to be destroyed. Required to keep the user interface responsive.
Visible Process	Involves any process that user is currently aware of, with significant consequences if the process is killed. Process within the paused state also need to be considered as it is possible that the component is still visible on the UI to the user. Services in the foreground are also affected as the systems it as if the user can visibly see this. Many of these processes can be controlled. Killing these processes would only occur if it were required to keep the rest of the foreground processes running.
Service Process	Processes that are not visible to the user, and generally responsible for the background processing. Largely, these processes are not killed unless the memory is required to keep the visible or foreground processes running.
Cached Process	Not currently needed processes. The system can kill these processes for additional resources; however, these processes are typically stored for efficient access between applications. Typically, this is where the other non-visible activity instances are stored, allowing for the overall user experience to not be affected if the process were killed. Previously stored states were also stored here

We would also want to introduce the idea of threads to ensure that the code is able to run concurrently, but also in sequential manor to avoid the existence of race conditions or improper process. For example, I call for a calculation to made in the model and then store the value to the database, I want to ensure that the calculation is fully complete before moving onto the database call. [17]

Synchronous	Also known as blocking or sequential programming. Requires for the previous action to be completed before moving onto the next task. Appropriate for processes that include dependencies or a specific sequence to be completed. Could lead to a decrease in responsiveness or performance, specifically for time-consuming tasks.
Asynchronous	Allows for tasks to be executed independently,

	improving performance. Utilises system resources more effectively and enhances effectiveness. Best for systems that requires significant real-time responsiveness or interactions. Significantly more complex to implement.
--	---

3.5 Market Research Report

Ensuring that there is an understanding of possible competitors showcases the ways that apps can find their place in the market and be a useful tool to its users. The ‘Star Student’ app largely utilises principles from project management tools to provide feedback and appropriate organisation for students, as well as providing some diary and wellness options for users as well. Therefore, I tried to find existing products that are still able to meet user’s needs in some capacity, however, have other features that I would like to improve upon.

3.5.1 Notion

Notion is an all-in-one productivity tool that allows users to use a series of predetermined blocks that can then be used for notetaking, project management, collaboration or documentation writing. Notion has a highly customisable format with significant options to build a workspace that is best for you. Many larger brands utilise this tool into their day to practise for their internal teams and processes but is also scalable for the everyday consumer.

From my personal experiences, many students, including myself, utilise Notion for their personal notetaking and overall organisation. However, unless you are happy using the basic features, the learning curve for even slightly more complex formatting and organisation is very steep. A huge amount of time is wasted setting up the initial workspace that can deter many users from using their workspace. [12]

Pros:

- Customisation
- Simple, clear layout for UI
- Cloud sharing
- Task and project management features
- Note taking features

Cons:

- Steep learning curve
- No offline mode

3.5.2 Evernote

Evernote is largely a note taking app, with options to make voice notes, sticky notes and handwritten notes, along with several ways to organise or sort. This app largely works as a paperless notebook, with cloud functionality so that it can be used from anywhere (so long as there is an internet connection). Newer versions of the app also have options for task management. [13]

This app generally covers some of the note taking options, so in my version, I would incorporate some more task/project management features.

Pros:

- Task management options
- Multiple ways to create notes
- Customisable widgets
- Integrations with other apps
- The organisation system
- Search feature

Cons:

- Paid version required for offline mode, and advanced features
- Limited task management features
- Limited calendar organisation features

3.5.3 Trello

Trello is the most direct piece of project management software that exists on the market. It largely has a focus on providing collaborative efforts to teams for their project goals. Instead, I would like to focus on creating a similar function but for individuals instead. [14]

To support this management, users have the options to create boards and cards, create checklists, set due dates, add main forms of labels, attach files directly to their tasks, and leave comments (which can still be useful in an individual environment.)

Pros:

- Flexibility in your task management
- Easy to learn and use
- Has a very visual layout, with multiple views

Cons:

- Unable to create detailed task dependencies
- Lack reporting features
- Some paid features

3.5.4 Forest

Forest is a studying app that uses pomodoro timers and gamification principles to encourage students to study in a healthy way. Additionally, there are several ways to categorise your study sessions to help with effectively organising your time. [15]

I believe combining the study tracking along with some of the foundational features of the task management seen in Trello or Evernote could make a more effective general use tool, as well as working for a larger demographic outside of just university students.

Pros:

- Options for ambient sound
- Clean UI layout
- Good organisation of tasks
- Gamification used to motivate players
- Widgets for quick support

Cons:

- Some main features are behind a pay wall
- Consistent manual input required to function

4 Implementation So Far (Early Deliverables)

4.1 Figma Prototype

To understand and outline some of the app functions, I have created a Figma prototype of how I expect the general UI to appear. Figma has significant features allowing for reusable components to be created, as well many integrations with Android, so many of the design options can be imported or reused within your project.

Along none of the true functions exist, there are still many options for navigation and animations. I have used this prototype to design the UI components and page layouts. While some changes will be made to the final version, this provided a good stepping stone for development.



Figure 4.1.1-4.1.4 Prototypes of the app's profile, planner, homepage and study centre (left to right)

4.2 Database Connection

In the current state of the app, a cloud service called Firebase is used to store the user's information and can be accessed to ensure user accounts exist. Android and firebase have significant integrations allowing for seamless connectivity, alongside a lot of documentation, all that is required is an internet connection.

Here I ensure there is a stable internet connection before sending database queries through the documents, which are then stored to variables so then they can be verified within the model.

Below is my function responsible for checking if the user has access to the database:

```
class DatabaseInteractions{

    var resultDoc = arrayOf<String>()
    val applicationContext = CurrentApplication

    /* Checks the firebase database connection by checking if there is
    access to the test file.
    * */
    suspend fun checkDatabaseConnection(context: Context, collectionPath
: String, documentPath: String) {

if(NetworkConnectivity(applicationContext.instance.connectivityManager).t
estingInternetConnection()) {
    Log.d("FirestoreCheck", "Starting to connect to database")
    val db = Firebase.firestore
    val result =
        db.collection(collectionPath)
            .document(documentPath)
            .get()
            .await()
    if (result.exists()){
        Log.d("FirestoreDebug", "Firestore call succeeded")
        //Connection to the database has been made
        if (result == null) {
            //testing file cannot be found
            Log.d(ContentValues.TAG, "Testing file cannot be
found.");
            resultDoc = arrayOf("File is Empty")
        }else {
            //testing file has been found, saved values to a
variable
            Log.d(ContentValues.TAG, "${result.id} =>
${result.data}")
            resultDoc += arrayOf(result.id.toString(),
result.data.toString())
        }
    }

    Log.d("FirestoreDebug", "Firestore call completed")
    //Ensuring a result can be found
```

```

        if (result == null) {
            Log.d(ContentValues.TAG, "Testing file cannot be
found.");

            resultDoc = arrayOf("File is Empty")
        }else {
            //Present the values from the testing file that
can be compared
            Log.d(ContentValues.TAG, "${result.id} =>
${result.data}")
            resultDoc += arrayOf(result.id.toString(),
result.data.toString())
            Log.d(ContentValues.TAG,
resultDoc.contentToString())
        }

    }else{
        throw NoInternetConnection()
    }
}
}

```

4.3 UI Components

For the Star Student app to have an adaptable, consistent and modular design, I had decided to create base models for each of the UI components using Jetpack Compose, where the parameters can be used to tailor the object for a specific purpose. Each of these components utilise variable attributes such as their colour, to that later different themes and options can be set by the user.

Each of these components are used to populate each screen, therefore any design changes can be made and then cascade across the whole UI.

The structure general follows a pattern of container -> base object -> sub container. Below are the example for the text fields:

```

@Composable
fun textField(header: String, info: String) {
    Column() {
        Text(

```

```

        text = header,
        color = MaterialTheme.colorScheme.background,
        fontSize = 12.sp,
        modifier = Modifier
            .testTag("textFieldHeaderLabel")
    )
    Box(
        modifier = Modifier
            .background(
                shape = RoundedCornerShape(12f),
                color = MaterialTheme.colorScheme.tertiary
            )
            .size(45.dp, 8.dp)
            .padding(2.dp)
            .testTag("textFieldBackground")
    ) {
        Text(
            text = info,
            fontSize = 3.sp,
            color = MaterialTheme.colorScheme.primary,
            modifier = Modifier
                .testTag("textFieldInfoLabel")
        )
    }
}
}

```

4.4 Navigation

Rather than needing to set the content or UI for the activity, I instead have utilised a navigation class, that populates the Main Activity with the appropriate screen, with all that is required is that the NavController is sent around to ensure that the same instance is supported. Instead, the navigation class is called within the main activity and directs the user to each page.

An additional interface has been added between the navigation controller and the UI screen, provide a greater amount of clarity, but preventing direct interactions of the UI and navigation model.

Below is my code for the app's navigation:

```
@Composable
fun Navigation() {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination =
Screens.SignInRegisterScreen.route) {
        //Opening screen (Sign-in or register)
        composable( route = Screens.SignInRegisterScreen.route ) {
            SignInRegisterScreen( navController = navController)
        }
        //Sign in (Existing User)
        composable(route = Screens.SignInScreen.route)
        {
            SignInScreen(navController = navController)
        }

        //User enters their email
        composable(route = Screens.AddEmailScreen.route) {
            AddEmailScreen(navController = navController)
        }

        //Verify the use has access to their email
        composable(route = Screens.VerifyEmailScreen.route) {
            VerifyEmail(navController = navController)
        }

        //Redirect to the homepage
        composable(route = Screens.HomePageScreen.route) {
            //
            Homepage(navController = navController)
            Homepage()
        }

        //Saves user's new passwords to the database and creates a
new user
        composable(route = Screens.CreatePasswordsScreen.route) {
            CreatePassword(navController = navController)
        }
    }
}
```

```

    }
}

```

Here is one of the exceptions that are thrown and then have the message represented within a pop-up:

```

/* Exception throw when there is no internet connection*/
class NoInternetConnection()
    : Exception("No Internet Connection. Please try again later.")

```

4.5 ViewModels

The ViewModels are responsible for monitoring the UI for possible state changes and providing the response from the database when required. Each ViewModel is connected to a corresponding screen. This involves having a series of state variables can be updated and represented on the UI screens. This largely involves a series of setter and getter methods that are triggered by the user's actions.

This also where the models would be instantiated and referenced, so that the UI doesn't need to be directly interacted with. Some coroutines and synchronous principles where required, as the model is unable keep up with the quick UI response. This is where loaders are used, as it gives user some feedback for the system status.

All the methods for the button are also stored here, as then the coroutines can be maintained and easily updated.

Below as some of the ViewModels used by the Sign in and Register Process:

```

/* ViewModel responsible for the Sign In page.
*/
class SignInViewModel : ViewModel() {

    private val email = Email()
    var emailString by mutableStateOf("")
    private set

```

```

var passwordString by mutableStateOf("")
    private set
var errorWindow by mutableStateOf(false)
    private set
var errorMessage by mutableStateOf("")
    private set

//Represent the email entered within the input field.
fun setEmailChange(emailValue: String){
    emailString = emailValue
}

//Represent the password entered within the input field.
fun setPasswordChange(passwordValue: String){
    passwordString = passwordValue
}

//Resetting the error window.
fun resetErrorWindow(){
    errorWindow = false
}

//Checks if the user's email is real, and then validated the
user account. If successful then move onto the homepage
fun next(navController: NavController){
    email.setEmail(emailString)
    try{
        if (email.checkRealEmail()){
            errorWindow = false
            viewModelScope.launch {
                val passwordValidation =
email.checkEmailExists(emailString, passwordString)
                if(passwordValidation){
                    navigateToHomepage(navController)
                }
            }
        }
    }catch(e: Exception){

```

```
        Log.d("TEST", "Error thrown")
        errorMessage = e.message.toString()
        resetErrorWindow()
        errorWindow = true
    }
}

//Buffer for checking the password
private fun Unit.await() {
    Log.d("AWAIT", "CHECKING PASSWORD....")
}

//Navigation the homepage
fun navigateToHomepage(navController: NavController){
    navController.navigate(Screens.HomePageScreen.route)
}

}

/* ViewModel for the sign in and register page.
*/
class SignInRegisterViewModel : ViewModel(){

    fun navigateToSignInScreen(navController: NavController){
        navController.navigate(Screens.SignInScreen.route)
    }

    fun navigateToAddEmailScreen(navController: NavController){
        navController.navigate(Screens.AddEmailScreen.route)
    }

}

/* ViewModel for the create password page.
*/
class CreatePasswordViewModel : ViewModel(){

    val password = Password()
```

```

        val userEmail =
CurrentApplication.instance.user.email.getEmail()

        var firstPassword by mutableStateOf("")
        private set
        var secondPassword by mutableStateOf("")
        private set
        var errorWindow by mutableStateOf(false)
        private set
        var errorMessage by mutableStateOf("")
        private set

        //Verify if the passwords are matching, if so then add the
        user to the database.
        fun verify(navController: NavController){
            try{
                if(password.checkPasswordsMatch(firstPassword,
secondPassword)){
                    viewModelScope.launch {

CurrentApplication.instance.user.addUserAccount(firstPassword).awa
it()

                    }
                    Log.d("USER", "Added New User")

navController.navigate(Screens.HomePageScreen.route)
                }

            }catch (e: Exception){
                Log.d("TEST", "Error thrown")
                errorMessage = e.message.toString()
                resetErrorWindow()
                errorWindow = true
            }
        }

        //Reset the error notification pop-up.
        fun resetErrorWindow(){
            errorWindow = false

```

```
    }

    //Represent the first password entered within the input field.
    fun setFirstPasswordValue(passwordValue: String){
        firstPassword = passwordValue
    }

    //Represent the second entered within the input field.
    fun setSecondPasswordValue(passwordValue: String){
        secondPassword = passwordValue
    }

    //Buffer added for connecting to the firebase database.
    private fun Unit.await() {
        Log.d("AWAIT", "DATABASE CONNECTION....")
    }

}

/* ViewModel for the add email view.
*/
class AddEmailViewModel() : ViewModel(){

    private val email = Email()
    var emailString by mutableStateOf("")
        private set
    var errorWindow by mutableStateOf(false)
        private set
    var errorMessage by mutableStateOf("")
        private set

    //Checks if the email enter is real, and then setting the user
    email to the account.
    fun next(navController: NavController){
        try{
            email.setEmail(emailString)
            if (email.checkRealEmail()){
```

```

        errorWindow = false

CurrentApplication.instance.setUser(email.getEmail())
        Log.d("TEST", email.getEmail())
        navigateToCreatePassword(navController)
    }
} catch (e: Exception) {
    Log.d("TEST", "Error thrown")
    errorMessage = e.message.toString()
    resetErrorWindow()
    errorWindow = true
}
}

//Navigate the create password screen.
fun navigateToCreatePassword(navController: NavController){

navController.navigate(Screens.CreatePasswordsScreen.route)
}

//Reset the error notification pop-up.
fun resetErrorWindow(){
    errorWindow = false
}

//Represent the email enter within the input field.
fun setEmailChange(emailValue: String){
    emailString = emailValue
}

}

//Buffer for loading
private fun Boolean.await() {
    Log.d("AWAIT", "CHECKING EMAIL STRING...")
}

```

4.6 Sign In/ Register Model

This model involves making calls to the Firebase database to check if our user exists and writing to the new user's information to the database for future logins. This has introduced the beginning of user accounts and a persistent user, which can later be changed to reduce the need to sign in every time.

Email checks and stored the values set for the user within the signing in process. We also can check if a user already exists within a system. Due to series of error that can be make, a toast pop-up has been implemented so that users can see possible mistakes and take actions to fix them.

There is a similar model for the password, however most of the model involves validation of the password provide, as a security risk can be posed if credentials are stored within the app. Additional security measures will be added in the future, such as hashed functions and hidden text fields.

Below are the Email and Password models, alongside their possible exceptions:

```
/* Model for the email information.
*/
class Email {

    private var email = ""
    private val applicationContext = CurrentApplication
    private val di = DatabaseInteractions()
    private var storedPassword: String? = ""

    //Check if the email contains an @ symbol, throws
    NotRealEmailAddress otherwise.
    fun checkRealEmail(): Boolean {
        if (containsAtSymbol()) {
            return true
        }
        throw NotRealEmailAddress()
    }

    //Checks the email for an @ symbol
    fun containsAtSymbol(): Boolean {
        return email.contains('@')
    }

    //Getter method for email
```

```

    fun getEmail(): String {
        return email
    }

    //Setter method for email
    fun setEmail(emailValue: String) {
        email = emailValue
    }

    //Checks if the email has been stored within the firestore
    database.

    suspend fun checkEmailExists(userEmail: String, userPassword:
    String): Boolean {
        val context = applicationContext.instance

        di.checkDatabaseConnection(context, "test",
        "testFile").await()
        if (di.resultDoc[0] != "File is empty") {
            //Queries the database
            val result = Firebase.firestore
                .collection("users")
                .whereEqualTo("username", userEmail)
                .get()
                .await()

            storedPassword = result.documents
                .firstOrNull()
                ?.getString("password")

            Log.d("TEST", "Password $userEmail")

            Log.d("TEST", "Password $storedPassword")
        }
        return comparePassword(userPassword, storedPassword)
    }

    //Compares the value of 2 passwords entered to see if they are
    the same.

    fun comparePassword(userPassword: String, realPassword:
    String?): Boolean {
        return userPassword == realPassword.toString()
    }

```

```
    }

    //Buffering for the database connection.
    private fun Unit.await() {
        Log.d("AWAIT", "DATABASE CONNECTION....")
    }

}

/* Exception throw when a false or fake email address has been
entered*/
class NotRealEmailAddress()
    : Exception("This is not a real email address. Please try
again.")

/* Involves functions relating to the user's password and it's
basic validation.
*/
class Password {

    private var password = ""

    //Checking if the passwords match.
    fun checkPasswordsMatch(firstPassword: String?,
secondPassword:String?): Boolean {
        if (firstPassword.toString() == secondPassword.toString())
        {
            return true
        }
        throw PasswordsDoNotMatch()
    }
}

/* Exception throw when the passwords entered do not match*/
class PasswordsDoNotMatch
    : Exception("Passwords do not match. Please try again.")
```

4.7 Blockers

Throughout this project, I have run into a series of blockers that significantly impacted the overall progress of the project. Although inevitable when completing a large-scale project, this greatly impacts the trajectory of the expected progress.

I will briefly go over the current issue, and the plans of mitigation or navigation for these issues.

4.7.1 Google Play Dependency

Originally, I had wanted to develop the app utilising some of the Google Play dependencies so that Star Student can later be sent to end users for testing or future production features. This also introduces a production cycle like professional app development groups.

After several attempts, a problem with the repository's permission was blocking the key required for Google Play to communicate with the project, therefore stopping any progress for this. The concept will likely be revisited, with an ADK file shared instead for manual downloads.

4.7.2 Email Delivery Service

I wanted to have an email service so user would be able to verify their email when registering, as this can be used as a further notification service, or give the option to reset passwords or accounts. However, when attempting to implement this, it proved to be a challenge.

Initially, I was attempting to find documentation directly from android developers, however I quickly learned that the only option was for a generated email that had to be manually sent by the user. Ultimately, this meant that the authentication would be useless and there is no way to directly contact the user.

An alternate showed using Firebase functions to send a browser call to an email service that can send an email on behalf of the app. This also ran into some issues as Firebase requires you to be on the paid plan to create custom functions that are deployable.

After this I tried to make a call directly to the email service, and while a connection was made, a 403-error kept occurring, which stated that the SMTP service blocks direct calls for mail prevention, with the work around also requiring payment.

In my research, other alternatives posed significant security risks or also require payment, with many of these efforts to scams and spam to be sent to users. Other possible options will be researched in the future, with a current basic solution for user verification.

Below is an example of the Kotlin function that was going to be used to send the email template:

```
class SendEmail {  
    // private var authCode : String  
    // private lateinit var context : Context  
    // private val senderEmail = "starstudent.contact@gmail.com"  
    // private lateinit var nc : NetworkConnectivity  
    //  
    // constructor(receivedAuthCode: String, context: Context){  
    //     authCode = receivedAuthCode  
    //     nc =  
    CurrentApplication.instance.applicationContext.NetworkConnectivity  
    ()  
    // }  
    //  
    // val fbfunc = FirebaseFunctions.getInstance()  
    //  
    // fun sendAuthEmail(recipientEmail: String){  
    //     if ()  
    //     fbfunc  
    //         .getHttpsCallable("sendEmail")  
    //         .call(mapOf("email" to email))  
    //         .addOnSuccessListener {  
    //             println("Email sent!")  
    //         }  
    //         .addOnFailureListener {  
    //             println("Error: ${it.message}")  
    //         }  
    // }  
}
```

5 Software Engineering

5.1 Requirement Elicitation

Understanding what user want and how they interact with the app is important, so I tried to create a series of user stories to understand what features can be useful and how I could best support them. Utilising some of the types of users defined with the paper related to gamification [7], I outlined three types of users; general user with typical behaviour, a user who becomes overwhelmed with large amounts of information, and a user who responses best from visual feedback.

Next, using my market research, I attempted to find gaps and features that could best support each user as well other users: planner section, customisation for the homepage, widget with icons and graphs.

Many of these concepts were feed into my Figma prototype, and then later became the requirements for the app's success. Next term, I plan to get a basic prototype to users to get some end-user feedback and understand if any major design changes need to be made.

This cycle will be repeated until the first version of the complete app is made to ensure that the user demographic effectively reflected, and the Star Student benefits its users.

5.2 Software Design

5.2.1 Process Flow

Below I have attached a diagram of the Star Students' current process flow:

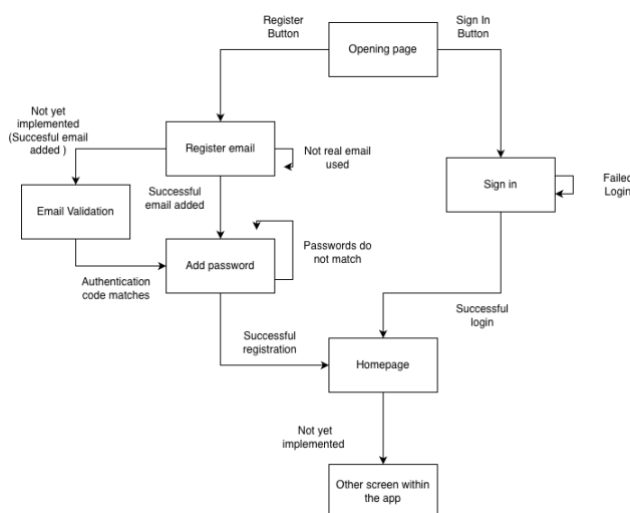


Figure 5.2.1.1 Current Process Flow Diagram

5.2 Testing

5.2.1 Unit and UI Testing

Largely my testing has consisted of ensuring that the UI components are present and behave in an expected way or using the action -> behaviour -> expected behaviour pattern, typically completed using the assertion functions. The UI and internet related tests posed an additional challenge due to the need for an emulator to run and assert many of these functions.

Further unit testing and performance-based testing will be completed to ensure that the app works in a specific way. One way to achieve this is to try and increase the overall test coverage of all the models and UI elements.

```
//Testing the render of the button component
@Test
fun testButtonExists() {
    composeTest.setContent {
        button("test") {}
    }
}

@Test
fun testButtonRender() {
    composeTest.setContent {
        button("test") {}
    }

    composeTest.waitForIdle()
    composeTest
        .onNodeWithTag("button")
        .assertIsDisplayed()
}

@Test
fun testDoesNotContainAtSymbol() {
    val testEmail = Email()
```

```
testEmail.setEmail("fakeEmail@gmail.com")
assertFalse(Email().containsAtSymbol())
}
```

5.3 Code Quality

5.3.1 Good Code, Code Smells and Refactoring

Comments, code structure and general code smells have been thought about and address as often as possible to ensure that the code is readable, easy to navigate and reduces the overall amount of dead/unreachable code. Many of these features are addressed once tested, as well as periodic refactoring to ensure that the code reaches a high enough standard. In the long term, this helps reduce the overall technical debt within the project, saving future time and resources.

5.3.2 MVVM Structure

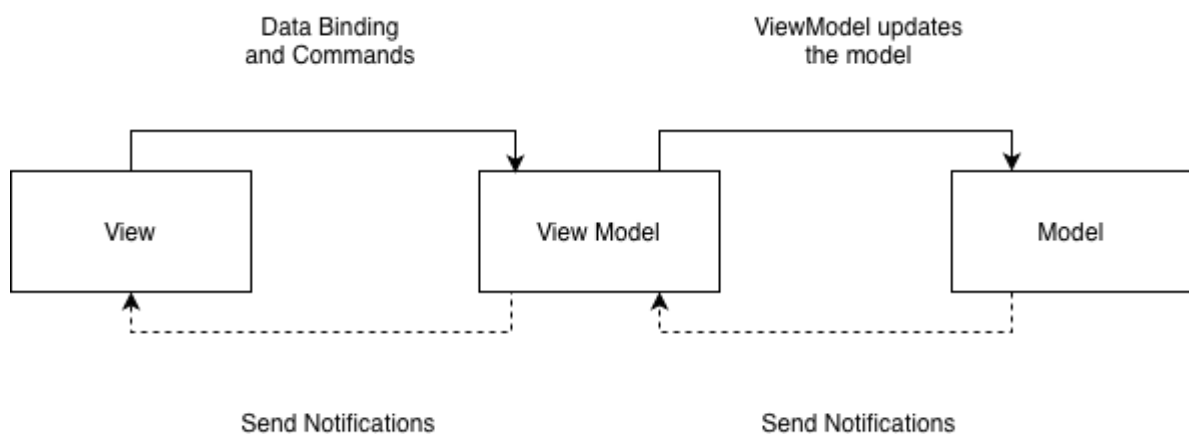


Figure 5.4.2.1 MVVM Model from Microsoft [10]

This is an architectural model responsible for ensuring that the business logic remains separated from the presentation logic. Ensuring that the view and model cannot directly interact with one another, allowing each part to be independently changed and tested without potential disruption to either side. Design and

development teams can work in parallel, with minimal issue from either side, allowing for the UI elements and model to be developed in parallel. Testing and later maintenance is also able to be more easily implemented within the segmented parts.

At a high level, the view communicates directly with the viewModel and viewModel communicates with the model, however the model is unable to communicate directly with the viewModel, and the view model is unable to communicate with the view. This allows for view to feed information into the viewModel and then receive the information for its display without further complication, with a similar idea existing for the viewModel and model.

Models typically return information that is not in a user-friendly format, so this model allows for the viewModel to receive the results from a model and put them into a format that can then be displayed for the UI which is then human readable.

Model Layer (Domain + Data layer)

Models are the logical non-visual classes responsible for any of the app's database or calculations. This typically involves anything related to database management, business logic or validations. Additional services or repositories would also be accessed via the model.

View (UI Layer)

The view should only be responsible for the visual representation that can be seen on screen. Sometimes this would be more complex with the introduction of animations or other UX features that improve the overall app experience, however no model logic should be present here. Instead, the view would contain the overall components that populate the screens visible to the user. The viewModel will then be built based on the components within the page and possible interactions from the user or object observers that would be available.

View Model (UI Layer)

The viewModel is responsible for binding the UI elements and actions to the model and its results. Any notifications or changes in state then need to be represented on the UI and provide feedback to the user. A series of the properties for the view can be defined within the viewModel; however the view is then responsible to represent any of this information. As mentioned before, the viewModel is responsible for formatting any data that has been received from the model to be represented on the view, unless further converters that might be needed. A series of interfaces are also available for any changed states for any properties that have changed or observers that have been monitored.

5.3.3 Design Patterns

Structural Pattern - Singleton

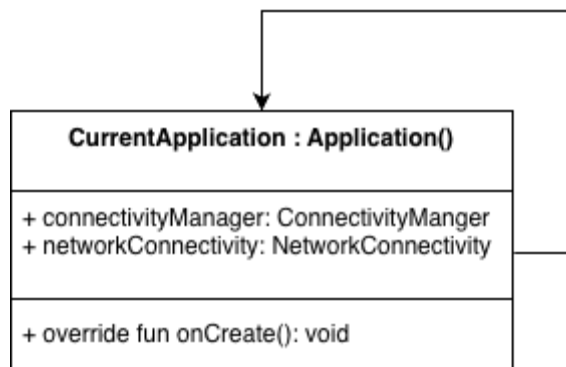


Figure 5.4.3.1 Example of the singleton implementation

A singleton design pattern needed to be introduced due combat to the potential issues related to data leaks. Without this pattern, several instances of the context or main activity could be created which would then cause instantiation errors. By introducing the singleton, there is now a centralised global point where the current instance can be referenced and interacted with, that has only one point of the app been created once while the app is opened.

While it is possible to send specific components into the arguments of the functions or classes, this is very risky and can lead to the original instance being lost or accidentally instantiated.

Behavioural Pattern - Observers

Observers are needed all over the app, due to the multiple points where the user can interact with the app and then requires feedback to be given. Therefore, this allows for the view to be informed about the changes from the model or user without the potential for high coupling.

The observable interface is where we instantiate and monitor the specific modules of the view, and then the observer interface defines the operation that is being notified. That way the observable can notify the observers once the current state has changed.

Possible benefits of this pattern include that the view components are not dependent directly on the result from the model to display the UI and can be updated whenever the model result changes. Constant communication between the model and view is not needed as the overall state would not differ unless a change has been recognised.

Architectural Pattern - MVVM Architecture

As mentioned before an MVVM architectural model is being used to decouple the view and model components of the code. The view and model can be updated concurrently and almost independently without significant consequence or further coordination.

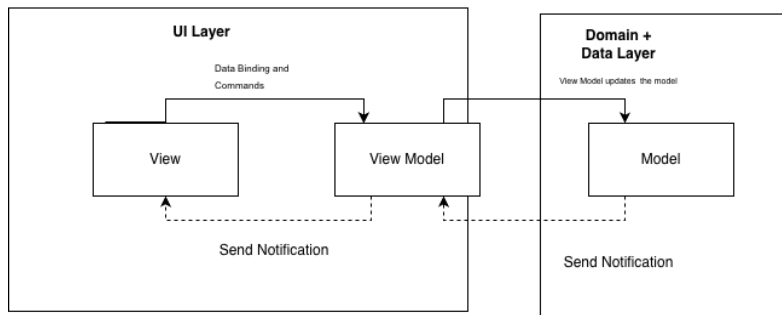


Figure 5.4.3.3 Example of the architectural implementation

5.4 Version Control

5.4.1 GitLab

This is the main form of version control that is used for my projects. All changes made to the code are committed and then immediately pushed to the remote repository. Therefore, if my laptop or computer were to fail, there would always be an up-to-date version of the project to continue from.

This also does not limit where I am able to continue developments, so long as a computer is accessible. This lessens the overall risk for hardware failure to affect progress of the project, regardless of how unlikely this is to occur.

5.4.2 Google Docs

Many of my reports are written initially using Google Docs, due to its portability and editing features. This allows for several drafts to be created, separated and can be accessed and edited from anywhere on any device, so long as there is an internet connection.

Final versions of these documents are then saved within my GitLab repository for organisation and future reference, as a failsafe measure.

5.4.3 Figma

Figma also has some version control features as your version history is stored to an account, which can be opened on a separate device. Like my reports, a final version is exported as PNGs and the original file to the GitLab remote repository as a failsafe measure.

7 Project Specification

7.1 Pre-requisites

- Android Phone
- App_debug.apk file -> PROJECT/product/app/build/outputs/apk/debug
- Security Permissions lowered to allow non-Google Play Apps

7.2 Installation instructions

1. Download App_debug.apk file.
2. Allow non-Google Play apps to run on your phone.
3. Open file from folder.
4. Trust Star Student
5. Ready to go!

7.3 Usage Instructions

- Register Account
- Sign in using credentials
- Navigate the Homepage

Bibliography

- [1] Dhabhar, F.S. Effects of stress on immune function: the good, the bad, and the beautiful. *Immunol Res* 58, 193–210 (2014). <https://doi.org/10.1007/s12026-014-8517-0>
- [2] Stallman, H. M., & Hurst, C. P. (2016). The University Stress Scale: Measuring Domains and Extent of Stress in University Students. *Australian Psychologist*, 51(2), 128–134. <https://doi.org/10.1111/ap.12127>
- [3] Asif, S., Muddassar, A., Shahzad, T.Z., Raouf, M. and Pervaiz, T. (2020). Frequency of depression, anxiety and stress among university students. *Pakistan Journal of Medical Sciences*, [online] 36(5), pp.971–976. [:https://doi.org/10.12669/pjms.36.5.1873](https://doi.org/10.12669/pjms.36.5.1873).
- [4] Martínez-Rubio D, Navarrete J, Montero-Marin J. Feasibility, Effectiveness, and Mechanisms of a Brief Mindfulness- and Compassion-Based Program to Reduce Stress in University Students: A Pilot Randomized Controlled Trial. *International Journal of Environmental Research and Public Health*. 2022; 19(1):154. <https://doi.org/10.3390/ijerph19010154>
- [5] Williams, T.S. (2025). Principles For the Design of Visual Progress Bars to Improve Motivation in Online Learning. [online] Beadle Scholar. Available at: <https://scholar.dsu.edu/theses/497> [Accessed 8 Oct. 2025].
- [6] Li, K. C., Lee, L. Y. K., Wong, S. L., Yau, I. S. Y., & Wong, B. T. M. (2018). Effects of mobile apps for nursing students: learning motivation, social interaction and study performance. *Open Learning: The Journal of Open, Distance and e-Learning*, 33(2), 99–114. <https://doi.org/10.1080/02680513.2018.1454832>
- [7] Oliveira, W., Hamari, J., Joaquim, S. et al. The effects of personalized gamification on students' flow experience, motivation, and enjoyment. *Smart Learn. Environ.* 9, 16 (2022). <https://doi.org/10.1186/s40561-022-00194-x>
- [8] Martins van Jaarsveld, G., Wong, J., Baars, M., Specht, M. and Paas, F. (2025). Enhancing goal attainment in higher education with a scripted conversational agent: Effects of monitoring and reflection support in digital learning. *Computers & Education*, 239, p.105441. doi:<https://doi.org/10.1016/j.compedu.2025.105441>.
- [9] Firebase (2019). *Cloud Firestore* | *Firebase*. [online] Firebase. Available at: <https://firebase.google.com/docs/firestore>.
- [10] Stonis, M. (2022). *Model-View-ViewModel*. [online] learn.microsoft.com. Available at: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>.
- [11] Android Developers. (2019). *Understand the Activity Lifecycle* | *Android Developers*. [online] Available at: <https://developer.android.com/guide/components/activities/activity-lifecycle>.
- [12] Kantor, A. (2024). *My Honest Notion Review After Using It For +2 Years* (2025). [online] Thebusinessdive.com. Available at: <https://thebusinessdive.com/notion-review>.
- [13] Cao, J. (2024). *Evernote Review 2025: Is This Popular Note-Taking App Still Worth A Try? - The Upbase Blog*. [online] The Upbase Blog. Available at: <https://upbase.io/blog/evernote-review/> [Accessed 19 Dec. 2025].
- [14] The (2025). *Trello Review: Kanban Software Review For 2025*. [online] The Digital Project Manager. Available at: <https://thedigitalprojectmanager.com/tools/trello-software-review/>.
- [15] By The Scales. (2021). *Is this app even worth it? Forest App Review*. [online] Available at: <https://bythescales.wordpress.com/2021/08/05/is-this-app-even-worth-it-forest-app-review/>.

- [16] Android Developers. (n.d.). *Processes and Application Lifecycle*. [online] Available at: <https://developer.android.com/guide/components/activities/process-lifecycle>.
- [17] GeeksforGeeks (2024). *Synchronous and Asynchronous Programming*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/javascript/synchronous-and-asynchronous-programming/>.
- [18] Chang, B.P.I., Webb, T.L., Benn, Y. and Stride, C.B. (2017). Which Factors Are Associated with Monitoring Goal Progress? *Frontiers in Psychology*, [online] 8. doi:<https://doi.org/10.3389/fpsyg.2017.00434>.

8 Appendices

8.1 Project diary

Week 13 - 15/12	Completed the interim report and connected the viewModel to the register and sign in views and	Catch up with development work.
----------------------------	--	---------------------------------

Week 12 - 08/12	Interim report and presentation	Complete a submission for the interim report
Week 11 - 01/12	- Implemented unit testing for model and UI elements. Jacoco dependency added to check for UI coverage. Readdresses pipeline issue -> still has not been resolved. - Created Interim report, presentation and retrospective for the end of term review.	Clean up project for the first term review, along with the user manual. Complete the ViewModels for current implementation.
Week 10 - 24/11	- Created initial implementation for the email verification model. Completed research about email sending implementation -> function needs to be readdressed. - Singleton class added to prevent potential dataleaks of the main activity.	Implement unit test cases for current implementation. Complete interim report, retrospective and presentation.
Week 9 - 17/11	- Completed research about architectural models and best practices. Currently utilising clean architecture methodology with the project using a MVVM structure for communication and separation. - Basic UI for the sign in and register pages completed, with some base of the model and viewmodels completed. Reusable UI components are also available outside of the currently completed ones. A navigation system has been implemented, and users are able to move around the sign in/ register portion of the app.	Create the user accounts and complete the model for sign in and register .
Week 8 - 10/11	Found cloud database options, and decided to use Firebase due to the volume of documentation and pre-existing integrations.	Developing the UI and navigation.
Week 7 - 03/11	Prototype created with Figma using process flow diagram previously created. Outlines the several features outlined in the project, and creates a possible layout for the UI.	Create the user accounts and connect the database.
Week 6 - 27/10	Researching into the database options available, restructuring plans to reflect the remaining time available.	Complete prototype and update requirements
Week 5 - 20/10	Completing further research about pipelines in Android apps. At this current state, Google play integration will not work, will be readdressed at a later date. Researching different solutions to address the security problems relating to the storage of user data. Created process flow diagram.	Complete implementation of basic UI features. (Placeholders if necessary)

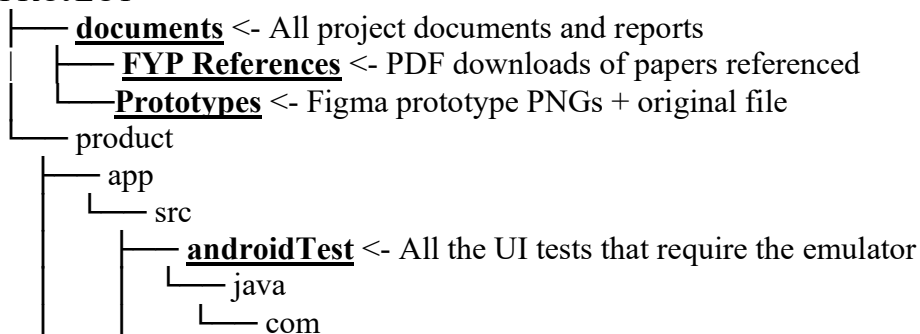
Week 4 - 13/10	Formatted the project repository for FYP, created the initial android studio project, updated the gitignore and began to create the pipeline, looking for possible testing methods and devices. Revised some software engineering best practices. Troubleshooting pipeline setup -> not able to resolve.	Fix pipeline issue, complete further research.
Week 3 - 06/10	- Created the initial project plan outlining the abstract, timeline, possible risks and mitigations, resources for future research. - Had first meeting with my supervisor, understood some of the missing features -> reworked the project plan requirements.	Set up project repository for future development -> Set up for SE practises to be included.
1.1.1 Week 2 - 29/09	Completing further research and brain-storming ideas for possible mobile apps, and understanding what can be useful.	Begin creating the project plan and requirements to meet.
1.1.2 Week 1 - 22/09	- Initial research about the projects available. - Understanding the specific project requirements and the format needed for the final year project.	Create a clear plan about a final decision for the project.

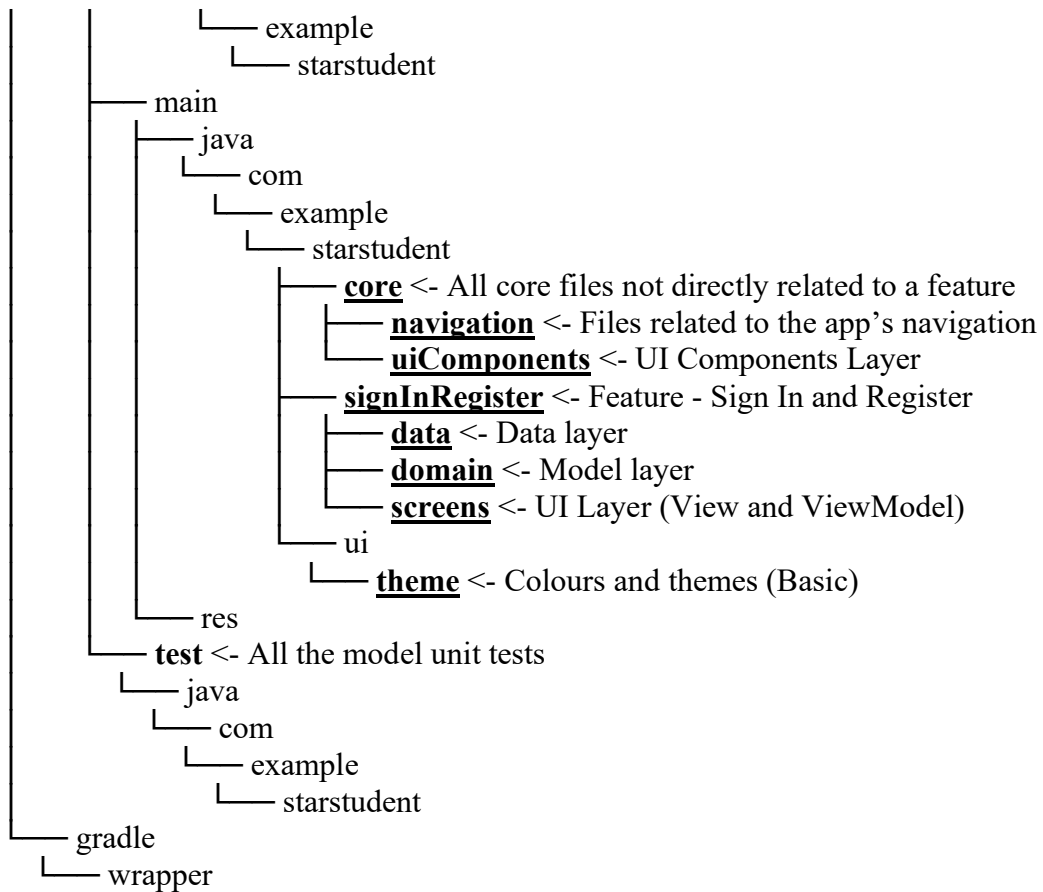
8.2 Structure of your Submission (Folder structure and how directories are organised)

All important directories have been highlighted and have been given a short description.

Project structure

PROJECT





9 Link to Video Demo

This video outlines the current features available on the star student app and some of the future features. Link - <https://youtu.be/A4DptcyXkE4>